

RELATÓRIO 4 - BUSCA SEQUENCIAL, MERGESORT E BUSCA BINÁRIA

Inserção de registros sem duplicidade em arquivo CSV

Disciplina: Análise de Algoritmos - Ciência da Computação - UCB

Grupo 5

Repositório: <https://github.com/branchescunha/algorithm-analysis/tree/main/report-04-search-and-sort>

1. Introdução

O trabalho implementa e compara duas soluções para inserir registros em um CSV sem duplicar uma chave de unicidade. A Solução 1 verifica duplicidade por busca sequencial, nas formas iterativa e recursiva. A Solução 2 mantém em RAM um índice ordenado por MergeSort e usa busca binária, também iterativa e recursiva. A análise foi construída para demonstrar formalmente as complexidades temporal e espacial e confrontá-las com medições experimentais explicitando todas as hipóteses usadas.

2. Base de dados e chave de unicidade

Foi utilizado `cnes_estabelecimentos.csv`, extraído do Cadastro Nacional de Estabelecimentos de Saúde (CNES) no Portal de Dados Abertos. A base possui 636.647 registros e 36 colunas. Uma validação sobre o arquivo integral confirmou que `CO_UNIDADE` possui 636.647 valores distintos e não está ordenada de forma crescente nem decrescente no arquivo original. Por isso, essa coluna atende simultaneamente aos dois requisitos do enunciado.

`CO_UNIDADE` é tratada como texto porque existem valores alfanuméricos. Em C, a igualdade e a ordem lexicográfica são avaliadas com `strcmp`. Como o comprimento máximo da chave é limitado e não cresce com N , o custo de uma comparação de chave é considerado constante na análise assintótica em função do número de registros.

O QUE	COMO	POR QUÊ
Base original	Leitura do CSV CNES em latin1, separado por ponto e vírgula.	Usar uma base pública real, como exigido.
Chave <code>CO_UNIDADE</code>	Validação de unicidade e de não ordenação no arquivo integral.	É única e não está ordenada, atendendo aos requisitos.
Base experimental	Amostra reproduzível de $N=1000$ registros, <code>random_state=42</code> , salva em UTF-8.	Controla N , evita stack overflow na busca recursiva e torna os testes repetíveis.

3. Hipóteses e desenho experimental

Para evitar ambiguidades entre teoria e experimento, as variáveis usadas são: N = quantidade de registros já existentes no destino; K = quantidade de registros recebidos no arquivo de entrada; D = quantidade de registros de entrada cuja chave já existe; I = quantidade de registros realmente novos. Nos testes principais, N inicia em 1000 e $D=I=K/2$.

Variável/condição	Valor nos testes	Como é garantido
N inicial	1000	<code>destino_backup.csv</code> é restaurado antes de cada execução.
K	10, 20, 40, 80	Quatro arquivos de entrada gerados por <code>gerar.py</code> .
D	$K/2$	Metade dos registros vem da própria base.
I	$K/2$	Metade vem de fora da amostra de 1000 registros.
Ordem do teste	Embaralhada	<code>sample(..., random_state=100+K)</code> .
Repetições de tempo	20	<code>benchmark.py</code> executa cada cenário 20 vezes e calcula a média.

A contagem de comparações é a principal evidência lógica, pois é determinística. O contador `long long` é incrementado exatamente quando `CO_UNIDADE` do vetor é comparada ao alvo. Os tempos são medidos com `clock()`; início e fim envolvem somente a operação analisada e a diferença é dividida por `CLOCKS_PER_SEC`. O arquivo físico é restaurado antes de cada repetição para impedir contaminação entre testes.

4. Solução 1 - Busca sequencial

4.1 Busca sequencial iterativa - demonstração

No pior caso, a chave não está no vetor. O laço testa todas as N posições. Se cada iteração possui custo constante c e há um custo fixo b :

$$T(N) = b + c + c + \dots + c \text{ (N vezes)} = b + cN = \Theta(N)$$

Logo, a busca sequencial iterativa é $O(N)$ e também $\Omega(N)$ no pior caso, portanto $\Theta(N)$. O algoritmo guarda apenas índices, contador e referências, todos em quantidade constante: $S(N)=c_s=\Theta(1)$.

4.2 Busca sequencial recursiva - demonstração

Cada chamada compara uma posição e avança para a próxima. No pior caso:

$$T(N) = T(N-1) + c$$

Substituindo repetidamente: $T(N)=T(N-2)+2c=\dots=T(N-k)+kc$. O caso base ocorre quando $k=N$; então $T(N)=T(0)+Nc=\Theta(N)$.

Para o espaço, cada chamada ainda não encerrada ocupa um frame da pilha: $S(N)=S(N-1)+f$. Expandindo da mesma maneira, $S(N)=S(0)+Nf=\Theta(N)$. Assim, a versão recursiva tem o mesmo tempo da iterativa, mas maior consumo de pilha.

4.3 Custo no experimento com K registros

Como gerar.py escolhe os primeiros $q=K/2$ registros da base como duplicados, essas buscas custam $1+2+\dots+q$ comparações. As q chaves novas não são encontradas e percorrem toda a base, que cresce uma posição após cada inserção: $N+(N+1)+\dots+(N+q-1)$. Somando:

$$C_{seq}(K,N) = q(q+1)/2 + qN + q(q-1)/2 = qN + q^2$$

$$\text{como } q=K/2: C_{seq}(K,N) = (K/2)N + K^2/4$$

Para $N=1000$, essa expressão gera exatamente 5025, 10100, 20400 e 41600 comparações para $K=10,20,40,80$.

5. Solução 2 - índice em RAM, MergeSort e busca binária

O CSV no disco não é ordenado. O programa carrega apenas `CO_UNIDADE` para um vetor em RAM, ordena esse índice e faz as consultas sobre ele. Um registro aceito é escrito no fim do CSV com `append`; somente a chave é inserida na posição correta do vetor em RAM. Assim, a exigência de manter o arquivo físico desordenado é preservada enquanto a busca binária recebe uma estrutura ordenada.

5.1 MergeSort recursivo - demonstração temporal

Cada chamada divide o problema em duas metades e a intercalação das duas metades percorre $\Theta(N)$ elementos:

$$T(N) = 2T(N/2) + cN$$

Após j expansões: $T(N)=2^jT(N/2^j)+jcN$. A divisão termina quando $N/2^j=1$, logo $j=\log_2 N$. Substituindo:

$T(N)=N \cdot T(1)+cN\log_2 N=\Theta(N \log N)$. Essa demonstração vale para melhor, médio e pior caso, pois a estrutura de divisões é a mesma.

Espaço: a profundidade recursiva é $\log_2 N$, portanto a pilha custa $\Theta(\log N)$. Durante merge, os vetores temporários contêm ao todo $\Theta(N)$ elementos no maior nível. Como $\Theta(N)$ domina $\Theta(\log N)$, o espaço auxiliar total é $\Theta(N)$.

5.2 MergeSort iterativo - demonstração temporal

A versão iterativa intercala blocos de tamanho 1,2,4,8,... até atingir N . O número de passagens p satisfaz $2^p \geq N$, portanto $p=\lceil \log_2 N \rceil$. Em cada passagem, todos os N elementos participam de intercalações:

$$T(N) = N \cdot \lceil \log_2 N \rceil = \Theta(N \log N)$$

Não há pilha recursiva, mas merge continua usando vetores temporários de tamanho total $\Theta(N)$ no pico. Portanto o espaço auxiliar também é $\Theta(N)$.

5.3 Busca binária iterativa e recursiva - demonstração

Após cada comparação, resta no máximo metade do intervalo. Depois de k passos, o tamanho máximo é $N/2^k$. O caso base ocorre quando $N/2^k \leq 1$:

$$N/2^k \leq 1 \Rightarrow 2^k \geq N \Rightarrow k \geq \log_2 N$$

$$T(N) = T(N/2) + c = \Theta(\log N)$$

A versão iterativa reutiliza um número constante de índices, então $S(N)=\Theta(1)$. A recursiva mantém uma chamada por nível de divisão; como há $\Theta(\log N)$ níveis, $S(N)=\Theta(\log N)$.

5.4 Custo global e comparação das soluções

Para K verificações, a Solução 1 tem custo de busca $\Theta(KN)$ quando K é pequeno em relação a N ; considerando o crescimento da base pelas inserções, o limite geral inclui $\Theta(K^2)$. Na Solução 2, a ordenação inicial custa $\Theta(N \log N)$ e as K buscas custam $\Theta(K \log N)$. Contudo, esta implementação mantém o vetor ordenado deslocando elementos após cada inserção válida, custo $\Theta(N)$ no pior caso por inserção. Para I inserções:

Solução 1: $\Theta(KN + K^2)$

Solução 2 completa: $\Theta(N \log N + K \log N + I \cdot N)$

Se o cenário for predominantemente de consultas, ou se a estrutura ordenada for mantida por uma estrutura de inserção mais eficiente, a vantagem da busca binária torna-se ainda mais relevante. Aqui, o termo $I \cdot N$ é explicitado para não confundir complexidade da busca com complexidade da solução completa.

6. Validação experimental da teoria

6.1 Experimento principal: $N=1000$ e K variável

K	N_0	D	I	Seq. teoria	Seq. obs.	Bin. obs.	Bin. limite
10	1000	5	5	5025	5025	99	100
20	1000	10	10	10100	10100	191	200
40	1000	20	20	20400	20400	377	400
80	1000	40	40	41600	41600	743	836

A tabela torna explícitas as hipóteses usadas no gráfico: $N_0=1000$ e $D=I=K/2$. Na busca sequencial, teoria e experimento coincidem exatamente em todos os pontos (diferença zero). Na binária, a coluna “limite” é o máximo de comparações permitido pela redução pela metade em cada consulta, considerando o crescimento do índice após inserções; os valores observados ficam sempre abaixo desse limite.

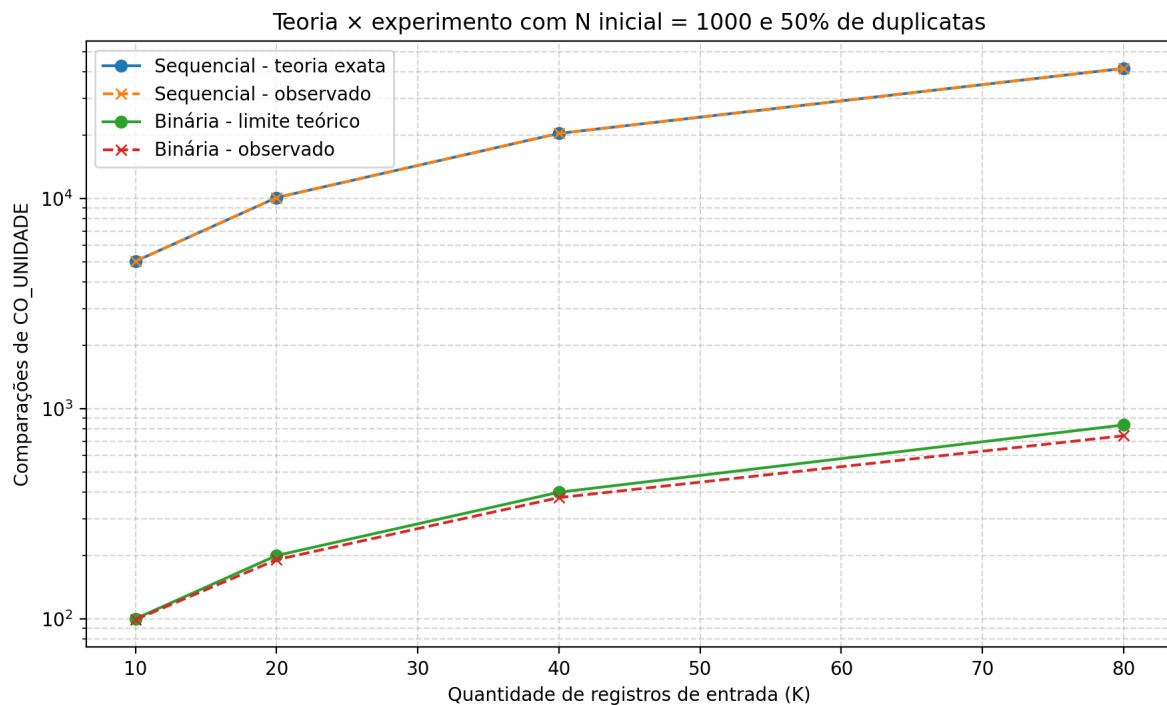


Figura 1 - Teoria x experimento no workload principal. A escala logarítmica permite visualizar as duas ordens de grandeza.

Na Figura 1, as curvas de teoria e experimento da busca sequencial se sobrepõem. Para $K=80$, $C_{seq}=41600$ tanto na fórmula quanto na medição. A busca binária realiza 743 comparações, abaixo do limite teórico de 836 para a sequência concreta do teste. Portanto o gráfico não é apenas ilustrativo: cada curva está ligada aos valores e hipóteses apresentados na tabela.

6.2 Experimento controlado variando N

Para observar diretamente a dependência em N, foi executado um segundo experimento com uma única chave ausente e bases de N=125,250,500,1000. Uma chave ausente força a busca sequencial a percorrer todo o vetor e força a binária a seguir até o intervalo vazio. Foram feitas 50 repetições para os tempos; a quantidade de comparações é determinística.

N	Seq. teoria N	Seq. observada	Bin. teoria $\lfloor \log_2 N \rfloor + 1$	Bin. observada
125	125	125	7	7
250	250	250	8	8
500	500	500	9	9
1000	1000	1000	10	10

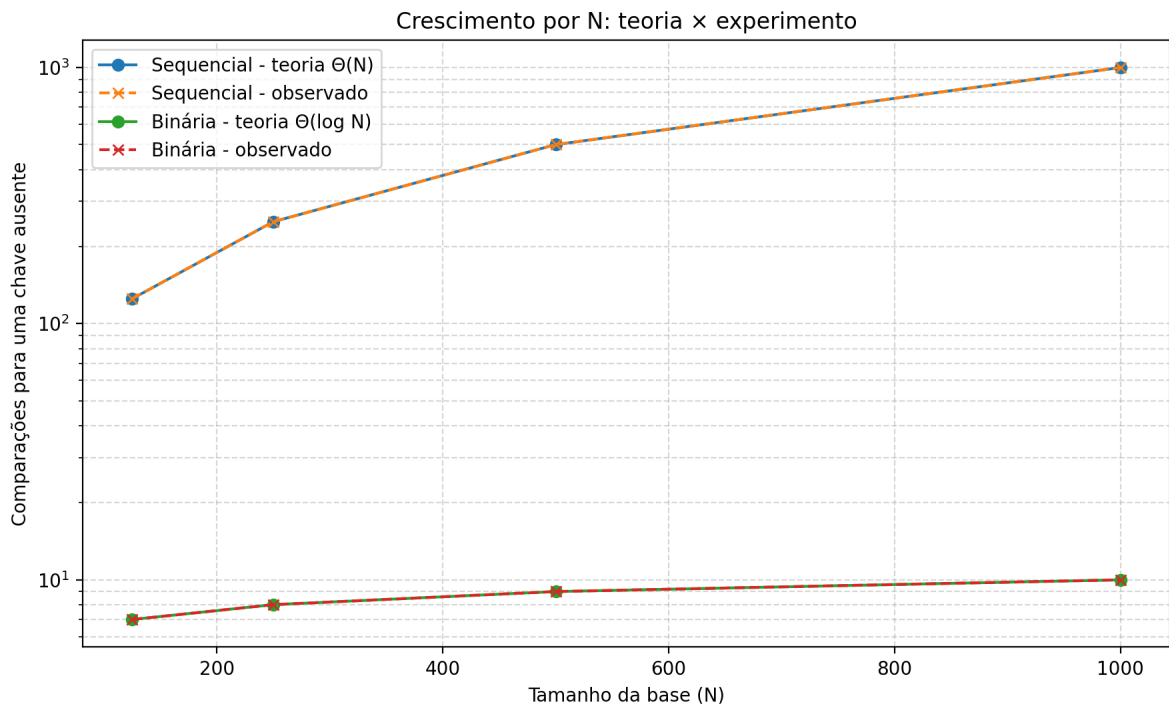


Figura 2 - Crescimento por N. A busca sequencial dobra as comparações quando N dobra; a binária acrescenta aproximadamente uma comparação.

A Figura 2 fornece a evidência experimental direta que faltaria se N permanecesse fixo: ao dobrar N de 125 até 1000, a sequencial passa de 125 para 1000 comparações (crescimento linear), enquanto a binária passa apenas de 7 para 10 (crescimento logarítmico). Os valores observados coincidem com o modelo teórico neste caso controlado.

6.3 MergeSort: teoria x tempo quando N varia

N	MergeSort iterativo (s)	MergeSort recursivo (s)
125	0.000082	0.000129
250	0.000180	0.000168
500	0.000285	0.000319
1000	0.000628	0.000657

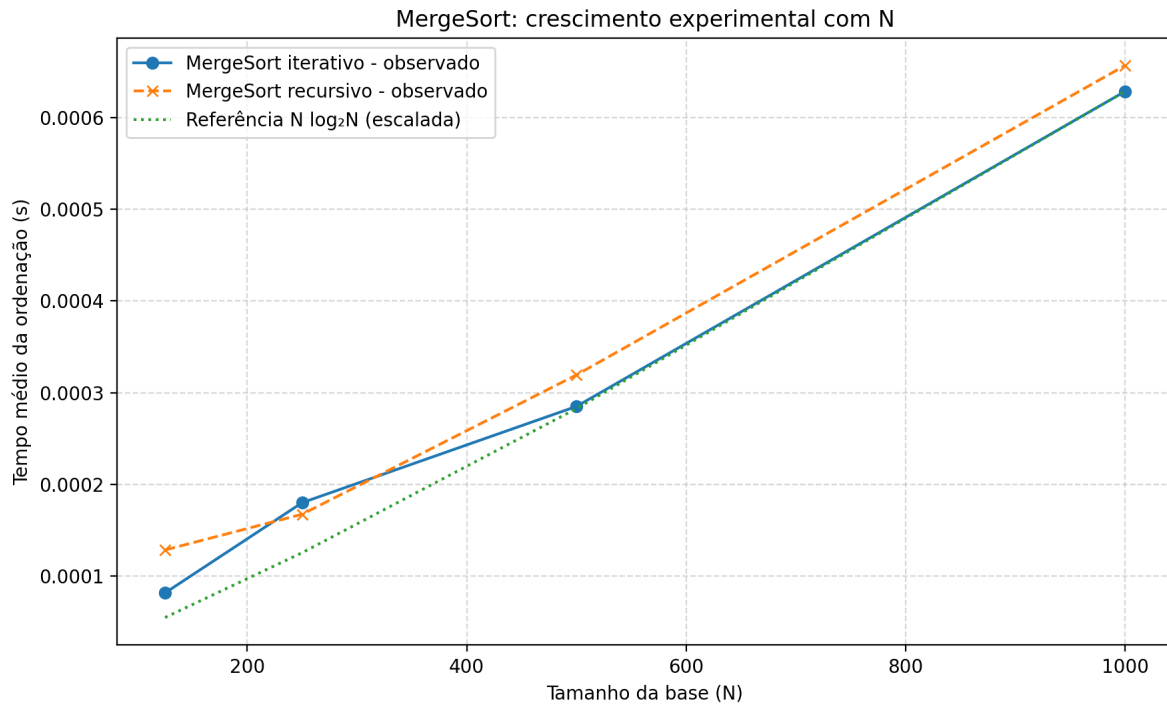


Figura 3 - Tempos médios da ordenação para N variável; a linha pontilhada é uma referência $N \log_2 N$ escalada.

A Figura 3 é usada apenas como validação de tendência, não como prova matemática. O tempo cresce com N e acompanha a forma $N \log N$ em ordem de grandeza. Como os tempos são inferiores a 1 ms, diferenças entre as versões e pequenas oscilações podem ser causadas pela resolução de `clock()`, cache e overhead constante. A demonstração formal de $\Theta(N \log N)$ permanece a evidência principal.

6.4 Tempos do workload principal

K	Seq. iter.	Seq. rec.	Sol. 2 iter.	Sol. 2 rec.
10	0.000046	0.000266	0.000625	0.000664
20	0.000090	0.000402	0.000678	0.000722
40	0.000130	0.000625	0.000636	0.000706
80	0.000256	0.001081	0.001277	0.001487

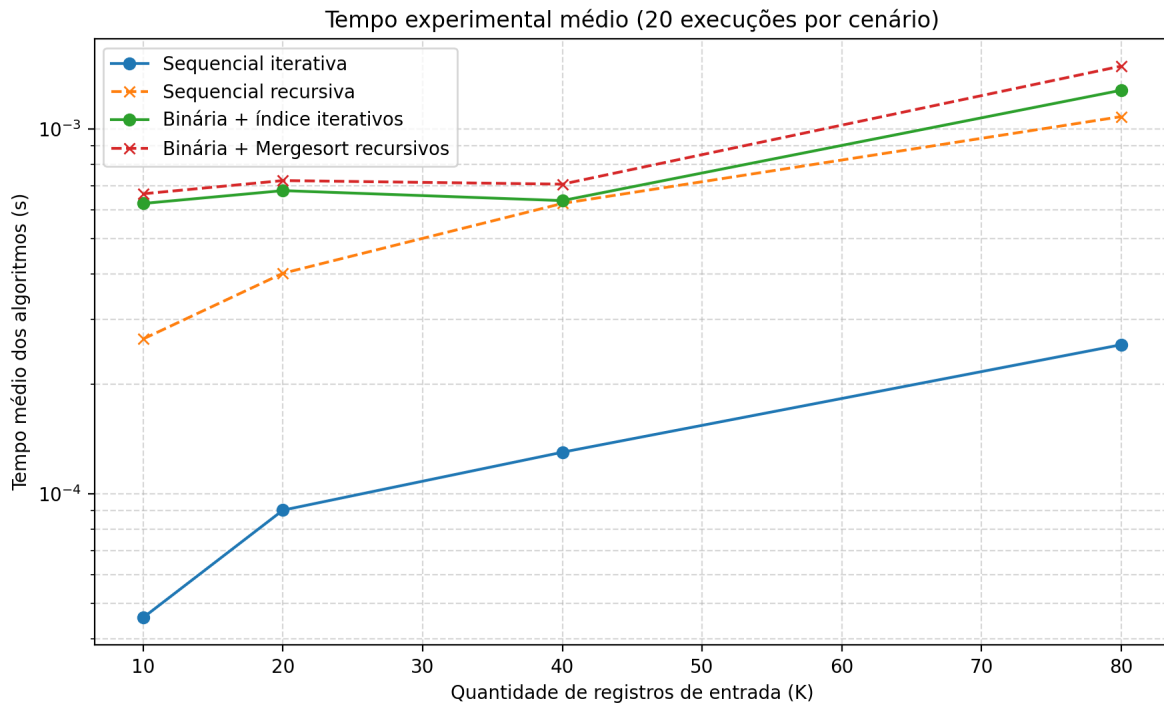


Figura 4 - Tempo médio do workload completo em 20 repetições por K.

A Figura 4 deve ser interpretada junto às complexidades. Com $N=1000$, ordenar e manter o índice adiciona um custo fixo/linear que pode fazer a Solução 2 ter maior tempo total em K pequenos, mesmo executando muito menos comparações de busca. Isso não contradiz $O(\log N)$: o gráfico mede a solução completa, não somente a função busca_binaria. A separação entre “tempo de busca”, “tempo de ordenação” e “tempo de manutenção” está registrada em estatisticas.csv.

7. Comparação de complexidade espacial

Algoritmo	Tempo	Espaço auxiliar	Demonstração/causa
Busca seq. iterativa	$\Theta(N)$	$\Theta(1)$	N iterações; quantidade constante de variáveis.
Busca seq. recursiva	$\Theta(N)$	$\Theta(N)$	Recorrência $S(N)=S(N-1)+f$ gera N frames.
MergeSort iterativo	$\Theta(N \log N)$	$\Theta(N)$	log N passagens; buffers de merge somam N no pico.
MergeSort recursivo	$\Theta(N \log N)$	$\Theta(N)$	Heap $\Theta(N)$ + pilha $\Theta(\log N)$; domina $\Theta(N)$.
Busca bin. iterativa	$\Theta(\log N)$	$\Theta(1)$	Intervalo é reduzido pela metade; índices constantes.
Busca bin. recursiva	$\Theta(\log N)$	$\Theta(\log N)$	Uma chamada por nível: profundidade log N.

Além desse espaço auxiliar, as duas soluções carregam as N chaves em RAM, portanto existe $\Theta(N)$ de armazenamento comum ao experimento. A tabela separa esse armazenamento da memória adicional específica do algoritmo.

8. Conclusão

CO_UNIDADE atende aos requisitos de unicidade e ausência de ordenação na base original. As quatro implementações impediram a introdução de duplicatas, e a Solução 2 preservou o CSV físico desordenado ao ordenar apenas o índice em RAM.

As demonstrações formais resultaram em $\Theta(N)$ para busca sequencial, $\Theta(\log N)$ para busca binária e $\Theta(N \log N)$ para MergeSort. As diferenças espaciais também foram demonstradas: a recursão aumenta o consumo de pilha para $\Theta(N)$ na sequencial e $\Theta(\log N)$ na binária; o MergeSort permanece $\Theta(N)$ devido aos buffers de intercalação.

Os experimentos sustentam essas conclusões. No workload com $N=1000$, a fórmula exata da sequencial coincidiu com todos os dados observados e a binária permaneceu abaixo de seu limite teórico. No experimento variando N , a sequencial cresceu de 125 para 1000 comparações enquanto a binária cresceu de 7 para 10, tornando visível a diferença entre crescimento linear e logarítmico. Os tempos de execução foram tratados com cautela por serem muito pequenos e por incluírem custos adicionais da solução completa.

9. Repositório público

O repositório contém os códigos em C, arquivos usados nos testes, scripts de preparação e benchmark, resultados CSV, gráficos e instruções de compilação e execução:

<https://github.com/branchescunha/algorithm-analysis/tree/main/report-04-search-and-sort>